

UNIT-V: Deep Learning for Time Series Forecasting

Jupyter Notebook: [Link](#)

1. Recurrent Neural Networks (RNNs)

RNNs are a class of neural networks designed for **sequence data**.

Key Idea: RNNs have a “memory” that captures information from previous time steps to influence predictions at the current step.

RNN Equation:

$$h_t = \sigma(w_h h_{t-1} + w_x x_t + b)$$

where,

h_t = hidden state at time t

x_t = input at time t

w_h, w_x = weights

σ = activation function

Python Example: Simple RNN for Time Series Forecasting

1. Simple RNN

```
[2]: # Sample data
data = np.sin(np.linspace(0, 50, 500))
X = []
y = []

# Create sequences (window size = 10)
for i in range(len(data)-10):
    X.append(data[i:i+10])
    y.append(data[i+10])
X = np.array(X).reshape(-1, 10, 1)
y = np.array(y)

# RNN model
model = Sequential()
model.add(SimpleRNN(50, activation='tanh', input_shape=(10, 1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Train
history = model.fit(X, y, epochs=20, batch_size=16, verbose=1)
```

Epoch 1/20

2025-10-24 10:29:14.594743: I tensorflow/core/grappler/optimizers: U is enabled.

31/31 ————— 2s 48ms/step - loss: 0.4732

Epoch 2/20

31/31 ————— 1s 45ms/step - loss: 0.0099

Epoch 3/20

31/31 ————— 1s 46ms/step - loss: 0.0057

Epoch 4/20

31/31 ————— 1s 45ms/step - loss: 0.0038

Epoch 5/20

31/31 ————— 1s 45ms/step - loss: 0.0026

Epoch 6/20

31/31 ————— 1s 48ms/step - loss: 0.0016

Epoch 7/20

31/31 ————— 1s 45ms/step - loss: 0.0012

Epoch 8/20

31/31 ————— 1s 47ms/step - loss: 0.0010

Epoch 9/20

31/31 ————— 1s 45ms/step - loss: 5.0964e-04

Epoch 10/20

31/31 ————— 2s 49ms/step - loss: 4.5919e-04

Epoch 11/20

31/31 ————— 1s 47ms/step - loss: 3.2280e-04

Epoch 12/20

31/31 ————— 1s 45ms/step - loss: 2.4490e-04

Epoch 13/20

31/31 ————— 1s 45ms/step - loss: 2.5520e-04

Epoch 14/20

31/31 ————— 1s 46ms/step - loss: 1.8422e-04

Epoch 15/20

31/31 ————— 1s 46ms/step - loss: 1.6427e-04

Epoch 16/20

31/31 ————— 1s 47ms/step - loss: 1.5407e-04

Epoch 17/20

31/31 ————— 1s 47ms/step - loss: 1.4204e-04

Epoch 18/20

31/31 ————— 1s 48ms/step - loss: 1.4124e-04

Epoch 19/20

31/31 ————— 1s 45ms/step - loss: 1.1720e-04

Epoch 20/20

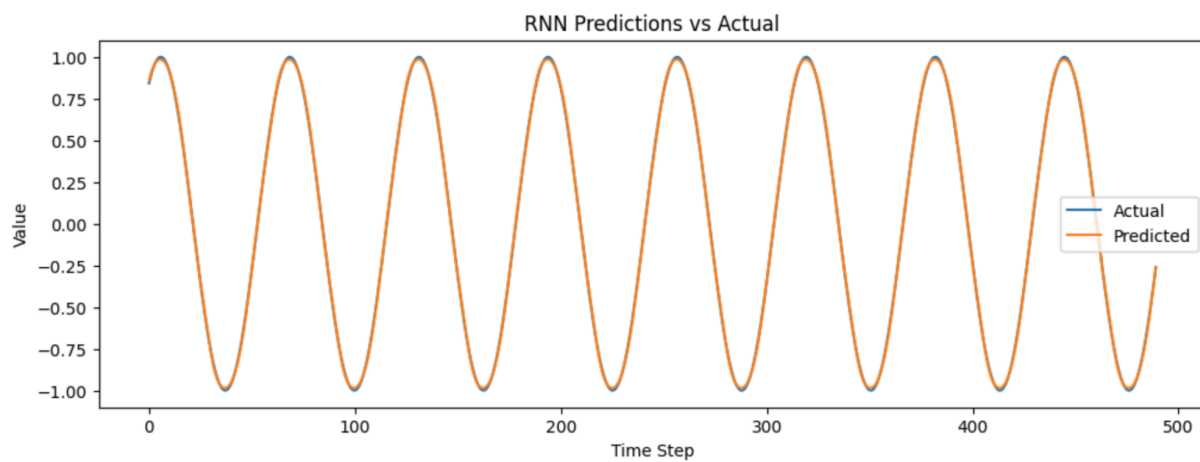
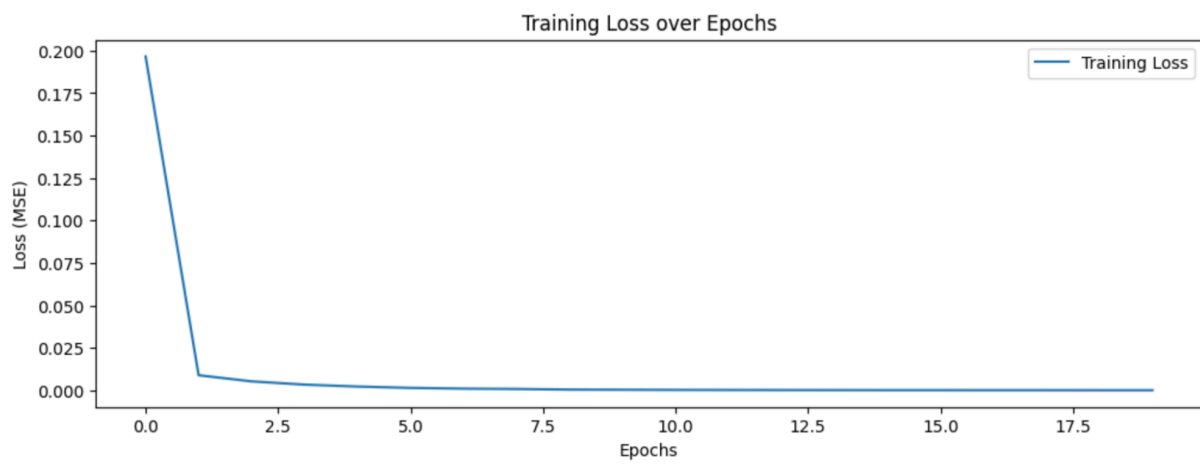
31/31 ————— 1s 46ms/step - loss: 1.1527e-04

Visualisation of Training Loss and RNN Prediction vs Actual

```
•[3]: # Predictions
y_pred = model.predict(X)

# --- Plot Loss Curve ---
plt.figure(figsize=(12,4))
plt.plot(history.history['loss'], label='Training Loss')
plt.title("Training Loss over Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.show()

# --- Plot Actual vs Predicted ---
plt.figure(figsize=(12,4))
plt.plot(y, label='Actual')
plt.plot(y_pred, label='Predicted')
plt.title("RNN Predictions vs Actual")
plt.xlabel("Time Step")
plt.ylabel("Value")
plt.legend()
plt.show()
```



2. Long Short-Term Memory (LSTM) Networks

LSTM is a **special type of RNN** that solves the **vanishing gradient problem** and can capture **long-term dependencies**.

LSTM Structure:

- **Forget Gate** – Decides what information to discard.
- **Input Gate** – Decides what new information to store.
- **Output Gate** – Determines output based on memory cell.

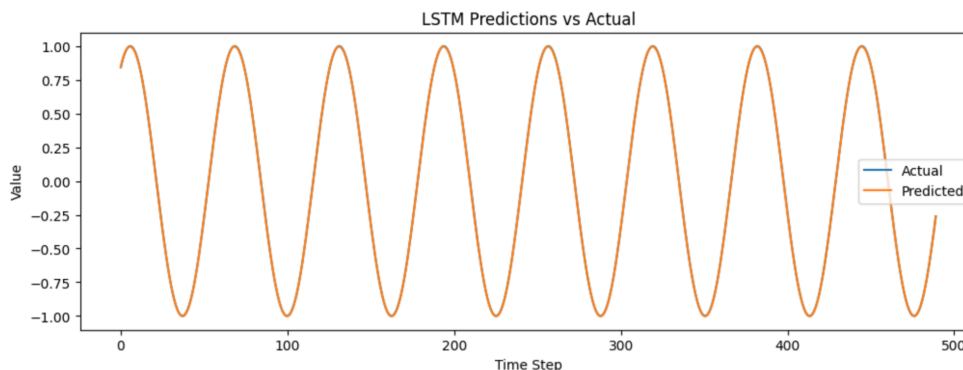
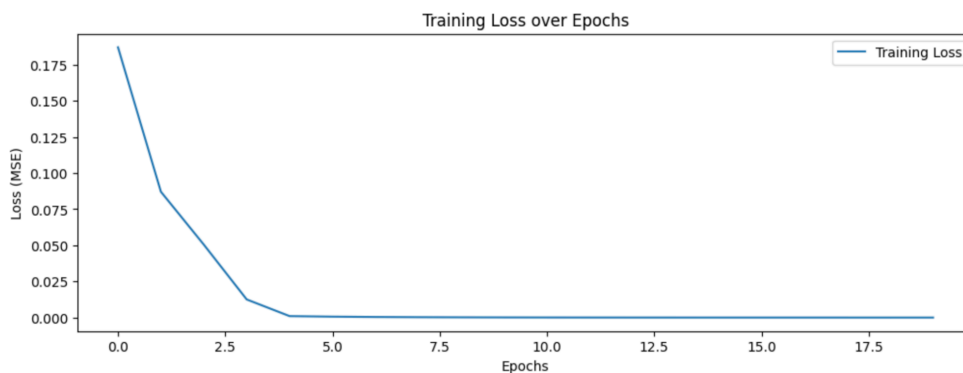
Python Example: LSTM for Time Series Forecasting

2. LSTM for Time series analysis

```
[7]: from tensorflow.keras.layers import LSTM

# LSTM model
lstm_model = Sequential()
lstm_model.add(LSTM(50, activation='tanh', input_shape=(10, 1)))
lstm_model.add(Dense(1))
lstm_model.compile(optimizer='adam', loss='mse')

#Train
lstm_history = lstm_model.fit(X, y, epochs=20, batch_size=16)
```



Advantages of LSTM:

- Remember long-term patterns.
- Reduces vanishing gradient problem.
- Effective for stock prediction, weather forecasting, energy consumption.

3. Hyperparameter Tuning and Optimization

Key hyperparameters for RNN/LSTM:

1. **Number of layers and neurons**
2. **Learning rate**
3. **Batch size**
4. **Sequence length / window size**
5. **Dropout rate** (to prevent overfitting)
6. **Optimizer** (Adam, RMSProp)

Optimization Techniques:

- **Grid Search** – Test all combinations.
- **Random Search** – Randomly sample hyperparameters.
- **Bayesian Optimization** – Probabilistic model for efficient search.

4. Model Evaluation and Validation

Performance Metrics for Time Series

$$\textcircled{1} \text{ MAE} = \text{Mean Absolute Error} = \frac{1}{n} \sum_{i=1}^n |Y_{\text{actual}} - Y_{\text{pred}}|$$

$$\textcircled{2} \text{ MSE} = \text{Mean Squared Error} = \frac{1}{n} \sum_{i=1}^n (Y_{\text{actual}} - Y_{\text{pred}})^2$$

$$\textcircled{3} \text{ RMSE} = \text{Root mean squared Error} = \sqrt{\text{MSE}}$$

$$\textcircled{4} \text{ MAPE} = \text{Mean Absolute Percentage Error}$$

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \frac{|Y_{\text{actual}} - Y_{\text{pred}}|}{Y_{\text{actual}}}$$

Cross-Validation Techniques

- **K-Fold Cross-Validation** (standard, not ideal for sequential data)
- **Time Series Split** (respects temporal order)

5. Practical Applications and Case Studies

Domains where Time Series Forecasting is widely used:

1. **Finance:** Stock price prediction, market trend analysis.
2. **Economics:** GDP forecasting, unemployment rate prediction.
3. **Healthcare:** Patient monitoring, disease spread forecasting.
4. **Energy:** Electricity demand, renewable energy prediction.
5. **Retail:** Sales forecasting, inventory planning.

Example: Predicting Stock Prices Using LSTM

- Collect historical stock data using **yfinance**.
- Preprocess data into sequences.
- Train LSTM model.
- Evaluate predictions using RMSE/MAE.

6. Conclusion

- RNNs and LSTMs are powerful for sequential data due to their memory capability.
- Feature engineering is less critical in deep learning compared to traditional ML.
- Hyperparameter tuning is essential for deep learning performance.
- Time series evaluation metrics must consider magnitude and percentage errors.
- Real-world applications span finance, healthcare, retail, and energy forecasting.

Flow Diagram of LSTM Forecasting Pipeline:

